

Balloon Burst Recursion

First of all, **your diagram is actually pointing toward the correct intuition**. The only thing that was missing was **why the parent's last balloon becomes the boundary of the child recursion**.

I'll explain using **your exact drawing**, because it is already very close.

Step 0 : Original Array

```
nums = [3,1,5,8]
```

We add virtual balloons.

```
Index : 0 1 2 3 4 5
Value : 1 3 1 5 8 1
```

Now we solve

```
solve(1,4)
```

which means

```
1 [3 1 5 8] 1
```

What does solve(i, j) actually mean?

This is the most important definition.

```
solve(i,j)
```

means

"Burst **only** the balloons from i to j."

Everything outside this interval

```
nums[i-1]
nums[j+1]
```

is **not touched**.

They stay alive until this recursive call finishes.

That is why they become the boundaries.

Your right-side diagram

On the right you drew

```
3 1 5 8
    ↑
```

and then

```
3 5 8
```

then

```
3 8
```

then

```
8
```

This is showing **the actual bursting order**.

Suppose recursion decides

```
Burst order
```

```
1
↓
3
↓
```

```
5
↓
8
```

Notice carefully...

The last balloon is

```
8
```

So while 8 is waiting,
everything before it disappears.
Eventually we reach

```
1 8 1
```

Only now do we burst 8.

So

```
coins = 1 × 8 × 1
```

NOT

```
5 × 8 × 1
```

because 5 is already gone.

That is exactly what you wrote on the right.

Now let's understand your LEFT TREE

You wrote something like

```
solve(8)
  /   \
solve(8) solve(3,1,5)
 /  \
1×8×1
```

Let's rewrite it a little cleaner.

```
solve(1,4)
Choose
k = 4
(last balloon = 8)
```

Immediately recursion says

```
Current coins
1 × 8 × 1
```

because

```
left boundary = nums[0]=1
right boundary = nums[5]=1
```

Now the important question is

"How do we make 8 become the last balloon?"

We must first remove

```
3 1 5
```

So recursion becomes

```
solve(1,3)
```

Exactly what you drew.

Now see both diagrams together

Your RIGHT SIDE says

```
Actual Bursting
3 1 5 8
↓
3 5 8
↓
3 8
↓
8
↓
(empty)
```

Your LEFT SIDE recursion says

```
solve(1,4)
↓
choose 8 last
↓
solve(1,3)
↓
choose 5 last
↓
solve(1,2)
↓
choose 1 last
↓
solve(1,1)
```

These are actually describing **the same process**, but in opposite directions.

RIGHT SIDE

Shows

Who is burst FIRST?

```
1
↓
3
↓
5
↓
8
```

LEFT SIDE

Shows

Who is chosen LAST inside every interval?

```
8
↓
5
↓
1
```

```
1
3
```

Notice something interesting.
They are opposite.
That is why Partition DP feels confusing initially.

Now the magical part

Let's zoom into the child recursion.
Parent is

```
1 [3 1 5 8] 1
```

Suppose

```
8
```

is the last balloon.

That means

```
8 is NOT burst yet.
```

It is waiting.

So recursion goes here

```
1 [3 1 5] 8
```

Notice what changed.

Originally boundary was

```
1 ..... 1
```

Now boundary became

```
1 ..... 8
```

This is the point where many people get confused.

Why is 8 still there?

Because

we promised that 8 will be burst LAST.

So until the child recursion finishes,

8 cannot disappear.

Now inside the child

We have

```
1 [3 1 5] 8
```

Suppose

```
5
```

is chosen last.

Again,

5 cannot disappear.

So recursion becomes

```
1 [3 1] 5
```

Notice what happened.

Earlier

```
right boundary = 8
```

Now

```
right boundary = 5
```

because

5 is waiting.

Exactly the same logic.

One more level

Now recursion is

```
1 [3 1] 5
```

Suppose

```
1
```

is chosen last.

Again,

1 waits.

Recursion becomes

```
1 [3] 1
```

Now boundaries become

```
1 ..... 1
```

Again,

because

```
1
```

is still alive.

This is exactly what your tree is doing

Your tree can be rewritten like this.

```
solve(1,4)
Boundaries
1 ..... 1
Choose last = 8
Coins
1x8x1
      |
      v
solve(1,3)
Boundaries
1 ..... 8
Choose last = 5
Coins
1x5x8
      |
      v
solve(1,2)
Boundaries
1 ..... 5
Choose last = 1
```

```
Coins
1x1x5
      |
      v
solve(1,1)
Boundaries
1 ..... 1
Choose last = 3
Coins
1x3x1
```

See how the boundary keeps changing.

```
Parent last balloon
↓
becomes
↓
Child boundary
```

This is exactly the pattern your notebook is trying to capture.

Why does the parent become the boundary?

Think of it like a promise.

Parent says

```
"I have decided
8
will die LAST."
```

If it must die last,
then while solving

```
3 1 5
```

it must still be alive.

Therefore

```
1 [3 1 5] 8
```

Similarly,
the child says

```
"I have decided
5
dies LAST."
```

Then while solving

```
3 1
```

5 must still exist.

So

```
1 [3 1] 5
```

Again,
the parent's last balloon becomes the child's boundary.

This is the mental picture I always use

Imagine every recursive call **freezes** its chosen last balloon.

```
solve(1,4)
Freeze
8
```

Now solve everything else.

```
1 [3 1 5] 8
```

Inside that,
freeze

```
5
```

Now solve

```
1 [3 1] 5
```

Inside that,
freeze

```
1
```

Now solve

```
1 [3] 1
```

Only after all children finish do we start bursting the frozen balloons:

```
3
↓
1
↓
5
↓
8
```

So recursion thinks in terms of **who survives until the end**, while the actual process is **who dies first**.

The one sentence to remember

A parent recursive call chooses a balloon to burst last. Since that balloon cannot be burst until the parent finishes, it stays alive during all child recursive calls and naturally becomes the left or right boundary of those child intervals.

Once you truly internalize this idea, the recurrence

$$\text{solve}(i, j) = \max_k (\text{solve}(i, k - 1) + \text{solve}(k + 1, j) + a[i - 1] \times a[k] \times a[j + 1])$$

stops looking like a formula to memorize—it becomes a direct consequence of the recursion tree you're drawing. In fact, your notebook is already visualizing this; the missing link was realizing that the **parent's "last" balloon is intentionally kept alive**, which is exactly why it appears as the child's boundary.

Absolutely. Let's build the **entire recursion tree** and **the real bursting process side-by-side**. This is exactly how I personally visualize Burst Balloons.

We'll use

```
nums = [3,1,5,8]
After adding boundaries
Index : 0 1 2 3 4 5
Value : 1 3 1 5 8 1
```

We start with

```
solve(1,4)
```

which represents

```
1 [3 1 5 8] 1
```

Legend

Every tree node will contain

```
solve(i,j)

Interval :
Current Balloons :
Boundaries :
Chosen Last Balloon :
Current Coins :
Left Subproblem :
Right Subproblem :
```

On the **right side**, we'll show the **actual bursting order** that corresponds to that recursive decision.

STEP 1

DP RECURSION

REAL WORLD

```
solve(1,4)

Interval :
[3 1 5 8]

Tree Node

Left Boundary = 1
Right Boundary = 1

Choose LAST

      8

Current Coins

1 × 8 × 1 = 8
```

```

      solve(1,4)
        |
    Last Balloon = 8
        /  \
    solve(1,3) solve(5,4)
    [3 1 5]   Empty
              = 0
  
```

Meanwhile, the real world is

```
1 [3 1 5 8] 1

We promised

8 will burst LAST.

So...

8 cannot disappear.

Only

3
1
5

may disappear.
```

Current alive balloons

```
1 3 1 5 8 1
```

Notice

```
8 is frozen.
```

STEP 2

Now recursion enters

```
solve(1,3)
```

The picture becomes

```
1 [3 1 5] 8
```


Notice
The right boundary changed!
It became

8

because 8 is still alive.

DP TREE

REAL BURST

`solve(1,3)`

Interval

`[3 1 5]`

Boundaries

Left =1

Right =8

Choose LAST

5

Current Coins

$1 \times 5 \times 8 = 40$

`solve(1,3)`

Last = 5

`solve(1,2)`

`[3 1]`

`solve(4,3)`

Empty

Real world

1 3 1 5 8

Again

5 is frozen.

Only

3

1

may burst.

Alive balloons

1 3 1 5 8

Frozen

5

8

STEP 3

Now recursion enters

`solve(1,2)`

Picture becomes

1 [3 1] 5

Notice
Boundary again changed.
Now

5

became the right boundary.
Because

5 has still not burst.

DP TREE

REAL BURST

`solve(1,2)`

Interval

[3 1]

Left Boundary =1

Right Boundary =5

Choose LAST

1

Coins

$1 \times 1 \times 5 = 5$

`solve(1,2)`

Last =1

`solve(1,1)`
[3]

`solve(3,2)`
Empty

Real world

Alive balloons

1 3 1 5 8

Frozen balloons

1
5
8

Only

3

may burst.

STEP 4

Now recursion enters

`solve(1,1)`

Picture

1 [3] 1

Notice

Boundary changed AGAIN.

Now

1

is the right boundary.

DP TREE

REAL BURST

`solve(1,1)`

Interval

[3]

Left Boundary =1

Right Boundary =1

Choose LAST

3

Coins

1 x3x1 =3

solve(1,1)

|

Last =3

/

\

Empty

Empty

Real world

Alive balloons

1 3 1 5 8

Frozen

3

1

5

8

Now there is nothing left.

So recursion starts returning.

NOW WATCH THE MAGIC

Everything now returns in reverse.

Return from solve(1,1)

Now

3

actually bursts.

Real world

1 3 1 5 8

↓

1 1 5 8

Return to solve(1,2)

Now

1

bursts.

Real world

1 1 5 8

↓

1 5 8

Notice

When 1 bursts

Neighbors are

1

5

Exactly why

1×1×5

was used.

Return to solve(1,3)

Now

5

bursts.

Real world

1 5 8

↓

1 8

Neighbors

1

8

Therefore

1×5×8

Return to solve(1,4)

Finally

8

bursts.

Real world

1 8 1

↓

1 1

Neighbors

1

1

Therefore

1×8×1

Entire Timeline

```
===== RECURSION =====
solve(1,4)
Choose LAST
8
↓
solve(1,3)
Choose LAST
5
↓
solve(1,2)
Choose LAST
1
```

```

↓
solve(1,1)
Choose LAST
3
↓
Return
↓
Burst
3
↓
Burst
1
↓
Burst
5
↓
Burst
8

```

Side-by-Side View

DP THINKING	REAL BURSTING
Choose LAST = 8	3 bursts
Choose LAST = 5	1 bursts
Choose LAST = 1	5 bursts
Choose LAST = 3	8 bursts

Notice something incredible.

The recursion goes **top-down** choosing the **last balloon**, but the actual balloons burst **bottom-up** as the recursive calls return.

The Boundary Inheritance (The Most Important Observation)

At each level, the parent's chosen last balloon is still alive, so it becomes the child's boundary:

```

Level 1
1 [3 1 5 8] 1
    ^
    Last = 8
Child receives
1 [3 1 5] 8

```

```

Level 2
1 [3 1 5] 8
    ^
    Last = 5
Child receives
1 [3 1] 5

```

```

Level 3
1 [3 1] 5
    ^
    Last = 1
Child receives
1 [3] 1

```

```

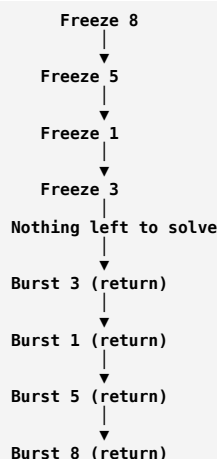
Level 4
1 [3] 1
    ^
    Last = 3

```

```
C++
solve(i, k-1)
+ solve(k+1, j)
+ nums[i-1] * nums[k] * nums[j+1]
```

because `nums[i-1]` and `nums[j+1]` are **the balloons that have been kept alive by the parent calls**. They act as the fixed neighbors when `k` is finally burst.

This is the mental model I recommend keeping:



Once you start thinking in terms of **"freeze the chosen last balloon, recurse on the remaining interval, then burst it while returning,"** the entire Partition DP becomes intuitive rather than something to memorize.

Great! Now that you've understood **why the recurrence works**, the code becomes much easier.

We'll implement both:

1. Memoization (Top-Down DP)
2. Tabulation (Bottom-Up DP)

using the same recurrence.

Recurrence

Suppose

```
C++
arr = [1,3,1,5,8,1]
```

Then

```
C++
solve(i, j)
```

means

Maximum coins obtainable by bursting only balloons from **i to j**.

The recurrence is

```
C++
solve(i, j) =
max(
    solve(i, k-1)
    + solve(k+1, j)
    + arr[i-1] * arr[k] * arr[j+1]
)
```

where

```
C++
i <= k <= j
```

1. Memoization (Top Down)

```
C++
class Solution {
public:
    int solve(int i, int j, vector<int>& nums, vector<vector<int>>& dp)
    {
        // No balloon left
        if(i > j)
            return 0;

        // Already calculated
        if(dp[i][j] != -1)
            return dp[i][j];

        int maxi = 0;

        // Try making every balloon the LAST balloon
        for(int k = i; k <= j; k++)
        {
            // Coins obtained by bursting k at last
            int currentCoins = nums[i-1] * nums[k] * nums[j+1];

            // Solve left interval
            int left = solve(i, k-1, nums, dp);

            // Solve right interval
            int right = solve(k+1, j, nums, dp);

            maxi = max(maxi, currentCoins + left + right);
        }

        return dp[i][j] = maxi;
    }

    int maxCoins(vector<int>& balloons)
    {
        int n = balloons.size();

        // Add virtual balloons
        vector<int> nums;

        nums.push_back(1);

        for(int x : balloons)
            nums.push_back(x);

        nums.push_back(1);

        vector<vector<int>> dp(n+2, vector<int>(n+2, -1));

        return solve(1, n, nums, dp);
    }
};
```

Memoization Tree

Suppose

```
solve(1,4)
```

tries

```
k=1
k=2
k=3
k=4
```

It will call

```
solve(2,4)
solve(3,4)
solve(4,4)
...
```

Many intervals repeat.

For example

```
solve(2,3)
```

can come from

```
solve(1,3)
and
solve(2,4)
```

Memoization stores

```
dp[2][3]
```

only once.

Time Complexity

There are

```
0(n^2)
```

states.

Each state tries

```
n
```

possible last balloons.

Therefore

```
0(n^3)
```

Space

```
0(n^2)
```

2. Tabulation (Bottom-Up)

Now we'll build the DP table ourselves.

Why do we build from smaller intervals?

Look at recurrence

```
C++
dp[i][j]
depends on
dp[i][k-1]
dp[k+1][j]
```

Notice

```
Left interval
```

is always smaller.

```
Right interval
```

is also smaller.

Therefore smaller intervals must already be computed.

Filling Order

Imagine

```

      j →
      1  2  3  4
i
1
2
3
4
```

The DP table is filled like this


```
Length =1
(1,1)
(2,2)
(3,3)
(4,4)
```

Then

```
Length =2
(1,2)
(2,3)
(3,4)
```

Then

```
Length =3
(1,3)
(2,4)
```

Then

```
Length =4
(1,4)
```

There are two common ways to implement this.

Method 1 (Recommended) — Interval Length

```
C++
class Solution {
public:
    int maxCoins(vector<int>& balloons)
    {
        int n = balloons.size();
        vector<int> nums;
        nums.push_back(1);
        for(int x : balloons)
            nums.push_back(x);
        nums.push_back(1);
        vector<vector<int>> dp(n+2, vector<int>(n+2, 0));
        // Length of current interval
        for(int len = 1; len <= n; len++)
        {
            // Starting index
            for(int i = 1; i + len - 1 <= n; i++)
            {
                int j = i + len - 1;
                int maxi = 0;
                // Choose every balloon as LAST
                for(int k = i; k <= j; k++)
                {
                    int currentCoins =
                        nums[i-1] * nums[k] * nums[j+1];

                    int left = dp[i][k-1];
                    int right = dp[k+1][j];
                    maxi = max(maxi,
                        currentCoins + left + right);
                }
                dp[i][j] = maxi;
            }
        }
        return dp[1][n];
    }
};
```

Method 2 (Reverse i Traversal)

This is the style used in Striver's DP series.

```
C++
class Solution {
public:
    int maxCoins(vector<int>& balloons)
    {
        int n = balloons.size();
        vector<int> nums;
        nums.push_back(1);
        for(int x : balloons)
            nums.push_back(x);
        nums.push_back(1);
        vector<vector<int>> dp(n+2, vector<int>(n+2, 0));
        for(int i = n; i >= 1; i--)
        {
            for(int j = i; j <= n; j++)
            {
                int maxi = 0;
                for(int k = i; k <= j; k++)
                {
                    int currentCoins =
                        nums[i-1] * nums[k] * nums[j+1];

                    int left = dp[i][k-1];
                    int right = dp[k+1][j];

                    maxi = max(maxi,
                        currentCoins + left + right);
                }
                dp[i][j] = maxi;
            }
        }
        return dp[1][n];
    }
};
```

Why does the reverse i traversal work?

Suppose we're computing

```
dp[2][4]
```

We need

```
dp[2][1]
dp[2][2]
dp[2][3]
dp[3][4]
dp[4][4]
dp[5][4]
```

Notice:

- `dp[i][k-1]` has a **smaller right boundary**.
- `dp[k+1][j]` has a **larger left boundary**.

When we iterate:

```
C++
for(int i = n; i >= 1; i--)
{
    for(int j = i; j <= n; j++)
```

- Rows with **larger i** (like `dp[3][4]`, `dp[4][4]`) are already computed because we're moving `i` downward.
- Values with the **same i** but smaller `j` (like `dp[2][2]`, `dp[2][3]`) are already computed because `j` increases from left to right.

So every dependency is available before it's needed.

Which version should you remember?

I recommend remembering:

- **Memoization** first (easier to derive).

- **Tabulation by interval length** because it directly reflects the idea of solving **smaller intervals before larger ones** and works for almost every interval DP problem (Burst Balloons, Matrix Chain Multiplication, Minimum Cost to Cut a Stick, Polygon Triangulation, etc.).

The reverse-i version is equally correct, but the interval-length version makes the dependency order more intuitive when you're first learning Partition DP.

ChatGPT can make mistakes. Check important info.